From regression to classification

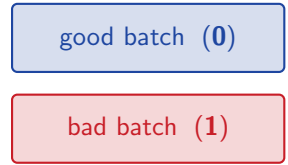
Until now you predicted a **number** (apartment rent). Now the target is a **category**:

Is this cheese batch bad?

Running example (the whole chapter): a factory makes 1000 cheese wheels a day; about **3% are bad** (contaminated / spoiled). Missing a bad batch is expensive.

The same shape of problem is everywhere:

- spam vs not-spam (email text)
- default vs repay (credit)
- churn vs stay (customers)
- sick vs healthy (screening)

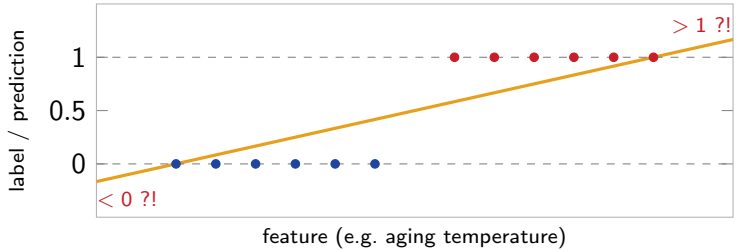


target $y \in \{0, 1\}$, not a number

The label is the rare "bad" class (positive, $\sim 3\%$).

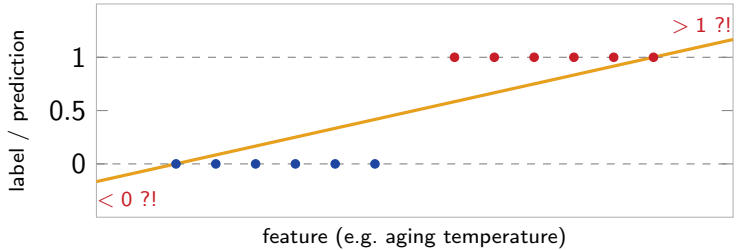
Predict-first: just fit a line to the 0/1 labels?

We know linear regression. Why not fit a straight line to labels that are 0 or 1, and read off the value?



Predict-first: just fit a line to the 0/1 labels?

We know linear regression. Why not fit a straight line to labels that are 0 or 1, and read off the value?



It breaks: predictions go **below 0 and above 1** (not probabilities), one far point **tilts the whole line**, and the implied threshold drifts. *We need a model whose output is squeezed into* $[0, 1]$ *– that is logistic regression.*

Outline

Binary and multiclass

Classification = assign each observation to one of a **finite set of unordered classes**. The target is categorical, not numeric.

Binary (2 classes)

- cheese: good / **bad**
- email: ham / spam
- loan: repay / default

Multiclass (> 2)

- digit: 0–9
- iris: setosa / versicolor / virginica
- sentiment: neg / neutral / pos

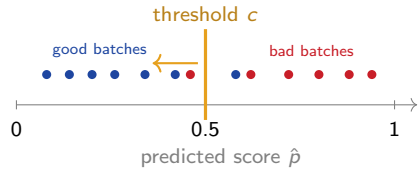
This deck = binary classification with logistic regression. More than two classes (one-vs-rest, softmax) is **L08**.

Probabilistic classifiers and the threshold

A good classifier outputs a **score** $\hat{p}(\mathbf{x}) \in [0, 1]$, not just a label. Threshold it into a decision:

$$\hat{y} = \begin{cases} 1 \text{ (bad)} & \hat{p} \geq c \\ 0 \text{ (good)} & \text{else} \end{cases}$$

Default $c = 0.5$, but c is a **choice**. Missing a bad batch hurts more than a re-check, so for cheese you may slide c *left* (catch more bad). How to pick it: **L07**.



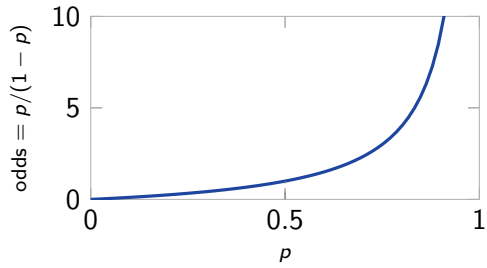
Right of $c \Rightarrow$ predict bad. Sliding c left catches more bad (with more false alarms).

Score vs probability. The number is a *score* that behaves like a probability; whether it is a *trustworthy* probability is a separate question (*calibration*). For now we use it to rank and threshold.

Odds

Odds $= \dfrac{p}{1-p}$ (p = probability of the positive class).

prob. p	odds	read as
0.5	1	1 : 1 (even)
0.75	3	3 : 1
0.9	9	9 : 1
0.1	1/9	1 : 9



This is betting odds. Bookmakers think in odds, not probabilities – "3 to 1" is the same $p/(1-p)$ (they quote odds *against*, $(1-p)/p$, shaded so the house wins). Probabilities live in $[0, 1]$; odds in $(0, \infty)$ – they *explode* as $p \to 1$. Not yet a free real number.

Log-odds (the logit)

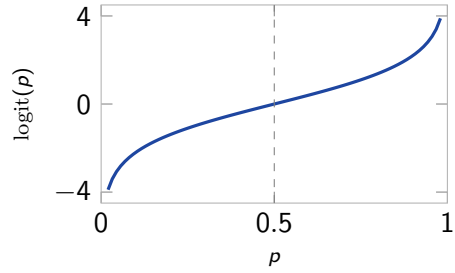
Take the log of the odds:

$$\operatorname{logit}(p) = \log \frac{p}{1-p} \quad \in \quad (-\infty, +\infty).$$

Now the value is a **free real number** – exactly what a linear model outputs.

Why log? Effects *add* on the log-odds scale. "Twice the odds" is always the same step $+\log 2$, whether you start near 0 or near 1. That additivity is what makes the coefficients interpretable later.

Symmetry: $\operatorname{logit}(p)$ and $\operatorname{logit}(1-p)$ are negatives of each other – $p = 0.5$ sits at 0.



Model the log-odds with a line

The whole idea of logistic regression in one line: assume the **log-odds is linear** in the features.

$$\log \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} \;=\; z \;=\; \theta_0 + \theta_1 x_1 + \cdots + \theta_p x_p \;=\; \boldsymbol{\theta}^\top \mathbf{x}.$$

We reuse everything from linear regression (a linear score $\boldsymbol{\theta}^\top \mathbf{x}$), but apply it to the log-odds instead of to y directly.

Predict-first: if the log-odds is a straight line in x, what shape does the *probability* $p(x)$ take?

Model the log-odds with a line

The whole idea of logistic regression in one line: assume the **log-odds is linear** in the features.

$$\log \frac{p(\mathbf{x})}{1 - p(\mathbf{x})} \;=\; z \;=\; \theta_0 + \theta_1 x_1 + \cdots + \theta_p x_p \;=\; \boldsymbol{\theta}^\top \mathbf{x}.$$

We reuse everything from linear regression (a linear score $\boldsymbol{\theta}^\top \mathbf{x}$), but apply it to the log-odds instead of to y directly.

Predict-first: if the log-odds is a straight line in x, what shape does the *probability* $p(x)$ take?

An **S-curve** – it has to bend to stay inside $[0, 1]$. Let us derive it.

Invert: the sigmoid

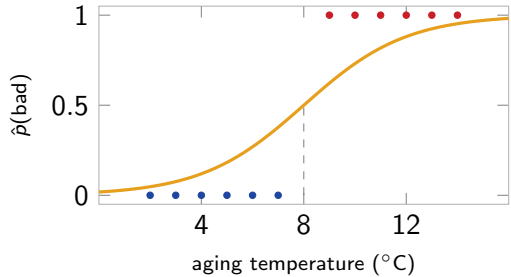
Start from the log-odds, **exponentiate** both sides, then solve for p:

$$\log \tfrac{p}{1-p} = z \;\Rightarrow\; \tfrac{p}{1-p} = e^z \;\Rightarrow\; p = (1-p)\,e^z$$
$$\Rightarrow\; p\,(1+e^z) = e^z \;\Rightarrow\; p = \tfrac{e^z}{1+e^z} = \tfrac{1}{1+e^{-z}} =: \sigma(z).$$

The **sigmoid** σ squashes any real score $z = \boldsymbol{\theta}^\top \mathbf{x}$ into a probability in $(0, 1)$:

- $z \to +\infty \Rightarrow p \to 1$
- $z = 0 \Rightarrow p = 0.5$
- $z \to -\infty \Rightarrow p \to 0$

This is logistic regression's hypothesis: $\hat{p}(\mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$.



Illustrative (balanced for clarity; real data is 3% bad).

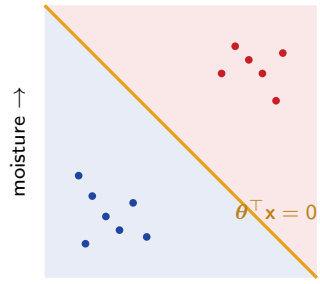
Same curve, "logistic growth": cumulative COVID cases in a wave trace a sigmoid – slow start, explosive middle, then a plateau as susceptibles run out.

Logistic regression is a linear classifier

Prediction: $\hat{p}(\mathbf{x}) = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$. Decide "bad" when $\hat{p} \geq 0.5$, i.e. when $\boldsymbol{\theta}^\top \mathbf{x} \geq 0$.

So the **decision boundary** $\boldsymbol{\theta}^\top \mathbf{x} = 0$ is a straight line (a hyperplane). The sigmoid only bends the *probability*, not the boundary.

Why is it called "regression"? It *regresses* a continuous quantity – the log-odds – on the features. The *fit* is regression-like; the *task* is classification.



Linear boundary; "bad" to the upper-right.

What the parameters do

- **Intercept** θ_0 shifts the S-curve left/right (sets the baseline level).
- **Weight** θ_j: its *magnitude* sets how steep the curve is in feature j; its *sign* sets the direction (raises or lowers p).
- Decision boundary: $\sigma(\boldsymbol{\theta}^\top \mathbf{x}) = 0.5 \iff \boldsymbol{\theta}^\top \mathbf{x} = 0$.

0.5 **is the default threshold, not a law.** It is just where $\boldsymbol{\theta}^\top \mathbf{x} = 0$. With asymmetric costs (missing bad cheese!) you will move it – **L07** shows how. The *model* gives the score; *you* pick the cutoff.

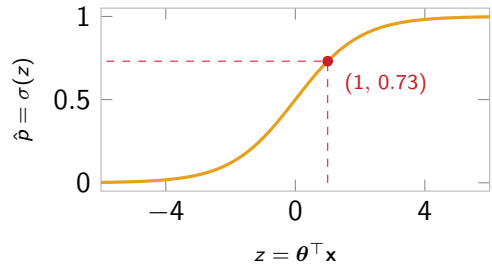
Worked numbers (one batch, by hand)

Fitted $\theta_0 = -4,\ \theta_1 = 0.5$ (one feature, aging temp x); take a batch at $x = 10°\text{C}$:

$$z = -4 + 0.5 \cdot 10 = \mathbf{1}$$

$$\hat{p} = \sigma(1) = \tfrac{1}{1+e^{-1}} = \mathbf{0.73}$$

$$\text{odds} = \tfrac{0.73}{0.27} = e^{1} = \mathbf{2.7}$$



Slope as an **odds ratio**: $e^{\theta_1} = e^{0.5} \approx \mathbf{1.65}$ – each $+1°\text{C}$ multiplies the odds of "bad" by 1.65 (+65%).

One z, one $\hat{p}$, one odds, one odds ratio – the whole mechanics in four numbers. *(Raw, unscaled features, so θ_1 reads "per °C".)*

Predict-first: why not squared loss?

We have a model $\hat{p} = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$. Could we just fit it by minimizing $\sum_i (y^{(i)} - \hat{p}^{(i)})^2$, like linear regression?

Predict-first: why not squared loss?

We have a model $\hat{p} = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$. Could we just fit it by minimizing $\sum_i (y^{(i)} - \hat{p}^{(i)})^2$, like linear regression?

Two problems:

- **Non-convex.** Squared loss composed with the sigmoid is not convex in $\boldsymbol{\theta}$ – gradient descent can get stuck in local minima.
- **Weak penalty.** If the truth is $y = 1$ and the model says $\hat{p} = 0.01$ (confidently wrong), squared loss is only ≈ 0.98. We want a loss that punishes *confident mistakes* hard.

We need a different loss – one built for probabilities.

Log-loss (cross-entropy)

Penalize by how much probability you gave the *true* label:

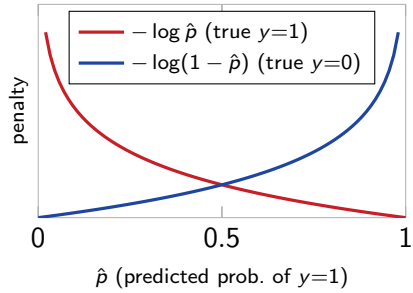
- true label $y = 1$: penalty $= -\log \hat{p}$
- true label $y = 0$: penalty $= -\log(1 - \hat{p})$

Both blow up to ∞ as you become confidently wrong, and go to 0 when you are confidently right.

One formula, where y picks the branch:

$$L(y, \hat{p}) = -\big[\, y \log \hat{p} + (1 - y) \log(1 - \hat{p}) \,\big].$$

This is **cross-entropy** / Bernoulli loss – the same loss reappears in neural nets and boosting (L11).



Where log-loss comes from

Two independent routes land on the *same* formula.

1. Maximum likelihood (statistics). The likelihood is the probability the model gives the labels we actually saw.

1. One row: $P(\text{observed label}) = \hat{p}_i^{\,y_i}(1-\hat{p}_i)^{1-y_i}$ ($\hat{p}_i$ if y_i=1, else $1-\hat{p}_i$ – the exponent switches a term on).
2. All rows (independent): multiply, $\prod_i \hat{p}_i^{\,y_i}(1-\hat{p}_i)^{1-y_i}$.
3. log + negate: $-\sum_i [\, y_i \log \hat{p}_i + (1-y_i)\log(1-\hat{p}_i)\,]$ = log-loss.

So fitting logistic regression = **maximum likelihood**.

2. Information theory. The very same quantity is the **cross-entropy** between the true labels and the model's predicted distribution – the average extra *bits* to encode the truth using the model's probabilities. Minimizing it pushes the predictions toward reality. (Hence the name "cross-entropy loss".)

Fitting logistic regression

We minimize the empirical risk (average log-loss):

$$\hat{\boldsymbol{\theta}} = \arg\min_{\boldsymbol{\theta}} \frac{1}{n} \sum_{i=1}^{n} -\left[y^{(i)} \log \sigma(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}) + (1 - y^{(i)}) \log(1 - \sigma(\boldsymbol{\theta}^\top \mathbf{x}^{(i)})) \right].$$

- **Convex** in $\boldsymbol{\theta}$ – no bad local minima.
- But **no closed-form** solution (unlike OLS) – fit **numerically** with an iterative optimizer (next slide). Convex $\Rightarrow$ it reaches the global optimum.

Separable-data trap. If a line perfectly splits the classes, the weights grow without bound (push $\hat{p} \to 0/1$ for more reward). The fix is the same as in regression: **regularization** (Ridge/Lasso, L03) – in sklearn this is the `C` parameter.

How is it actually optimized?

The log-loss is **convex and smooth** – we get a gradient *and* curvature to exploit.

- Plain **gradient descent** works (Chapter 1), but needs many steps.
- sklearn's default is **L-BFGS** – a *quasi-Newton* method: it uses curvature via an *approximate* inverse Hessian built from recent gradients (never the full $p \times p$ matrix), so it converges in far fewer iterations and stays memory-light. Convex + smooth is exactly where Newton-type methods shine.

solver	**when to use**
`lbfgs` (default)	medium, dense data; L2
`liblinear`	small data; supports L1
`saga`	large / sparse; L1 / elastic-net
`newton-cholesky`	$n \gg$ #features (exact Hessian)

Why not plain GD? On a convex, smooth loss, curvature (second-order) info reaches the optimum in a handful of iterations, not hundreds.

Logistic regression in sklearn

```python
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

clf = make_pipeline(
    StandardScaler(),                          # scale features (L01c)
    LogisticRegression(C=1.0, class_weight="balanced"),
)
clf.fit(X_train, y_train)
proba = clf.predict_proba(X_test)[:, 1]     # P(bad), bad=1
```

argument	what it controls
`C`	inverse regularization strength (small `C` = stronger penalty)
`penalty`	`"l2"` (default), `"l1"` (sparse)
`class_weight="balanced"`	up-weights the rare class – cheese is 3% bad

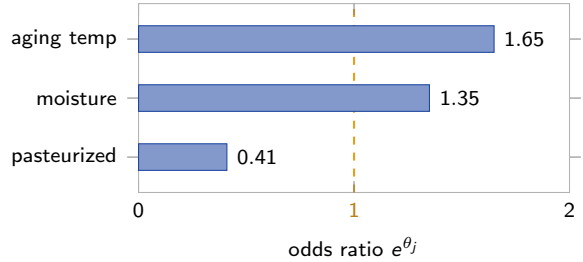
Scaling makes `coef_` per standard deviation; for per-unit odds ratios (next slides), fit on raw features.

Reading the coefficients

Still **very interpretable** – though less *directly* than linear regression:

- θ_j = effect on the **log-odds** per $+1$ unit of feature j.
- e^{θ_j} = the **odds ratio**: multiply the odds by this.

- aging temp: $e^{0.5} = 1.65$ ($+1°\text{C} \Rightarrow$ odds $\times 1.65$).
- moisture: 1.35 – wetter $\Rightarrow$ more likely bad.
- pasteurized: 0.41 – cuts the odds by $\sim 60\%$.



Orange line = 1 (no effect): bars right raise the odds, left lower them.

Mind the scale: in linear regression θ_j is the change in y itself; here it changes the **log-odds**, and the effect on the probability is non-linear (largest near $p = 0.5$). Read it as an odds ratio, not a direct effect. (Raw units; with `StandardScaler`, `coef_` is per standard deviation.)

Strengths, limits, and what's next

Strengths

- Interpretable (odds ratios).
- Outputs a probability/score.
- Fast; convex; great *baseline*.

Limits

- **Linear** decision boundary only.
- Assumes log-odds linear in features.
- Needs feature engineering (e.g. polynomial terms) for curved boundaries.

Two things to address next: how to *evaluate* a classifier honestly (next slide, then **L07**); and how to handle *more than two classes* (**L08 multiclass**).

Wait – how do we know it's any good?

We can *build* a classifier now. But how do we **score** it?

Predict-first: cheese is 3% bad. A lazy model that calls *every* batch "good" scores...

Wait – how do we know it's any good?

We can *build* a classifier now. But how do we **score** it?

Predict-first: cheese is 3% bad. A lazy model that calls *every* batch "good" scores... **97% accuracy** – while catching **zero** bad batches.

Accuracy is **misleading** on imbalanced data, and it ignores the asymmetric cost (a missed bad batch $\gg$ a needless re-check). Classification needs metrics that separate *"catches the bad ones"* from *"doesn't cry wolf"*: precision, recall, F1, ROC-AUC, PR-AUC – and the threshold c itself.

That is the whole next lecture – **L07: classification metrics**, on the same cheese factory.

Recap

- **Classification** predicts a category; a good classifier outputs a **score** in $[0, 1]$, then you **threshold**.
- **Odds** $= p/(1-p)$; **log-odds** maps $(0, 1) \to \mathbb{R}$.
- **Logistic regression** = a linear model on the log-odds, squashed to a probability by the **sigmoid** $\sigma(\boldsymbol{\theta}^\top \mathbf{x})$. Linear boundary.
- Fit by minimizing **log-loss** (cross-entropy) with gradient descent; regularize via `C`.
- Coefficients read as **odds ratios** (e^{θ_j}).

Workflow: scale features $\to$ fit logistic regression as your baseline $\to$ read the odds ratios $\to$ evaluate with the right metric (L07) $\to$ tune the threshold for your costs.